KOI Content Pack

Test Guide by Use Case

Seven use cases — what to do, what to expect, and why it matters

Document	KOI Content Pack — Test Guide by Use Case
Pack version	1.5.0
Guide revision	1.0
Applies to	Cortex XSIAM
Last updated	July 25, 2026

Work top to bottom the first time — later use cases assume earlier ones passed.

Contents

Before You Start

Need	Why it matters
Pack installed with demisto-sdk ... --xsiam	A zip upload lands the content but not working event collection.
KOI integration instance, test green	Everything except the Script Runner reads the KOI API.
An egress IP KOI accepts, or a Cortex engine that has one	KOI restricts its sensitive endpoints by source IP. A wrong egress returns HTTP 403 even with a valid key.
Core REST API integration enabled	KoiScanTracker pages endpoint groups past the 100-endpoint cap through it.

Check the engine before anything else. If the instance runs on a Cortex engine and that engine is disconnected, every koi-* command fails with "Engine ... is not connected" before it ever reaches KOI. That reads like an integration fault and is not one.

Generating test data on demand

You do not have to wait for real KOI activity. The KoiSimulateEvents automation (TestTools/KoiSimulateEvents) generates events shaped exactly like real ones and reports the answers in advance, so you check arithmetic rather than impressions.

```
!KoiSimulateEvents alerts=40 duplicate_factor=8 audit=60
```

It writes to koisim_koisim_raw, never koi_koi_raw, so it cannot corrupt real data or a second project's measurements. Query that dataset, or substitute its name into any query you are testing.

1. Event Collection

Why: everything downstream reads koi_koi_raw. If collection is wrong, every later result is wrong in a way that looks like a content bug.

Do

1. Enable Fetch events on the instance and wait two cycles.
2. Run:

```
dataset = koi_koi_raw | comp count() as rows by source_log_type
```

Expect

- Rows under both Alerts and Audit.

Then check the counts mean what they say

```
dataset = koi_koi_raw | filter source_log_type = "Alerts"
| alter nid = json_extract_scalar(metadata, "$.notification_event_id")
| comp count() as rows, count_distinct(nid) as real_alerts
```

rows will be far larger than real_alerts. That gap is normal — KOI re-sends every still-open alert on each fetch. notification_event_id is the only per-alert identity; koi_event_id is the scan batch and finding_uid is the policy definition. Anything counting alerts must use count_distinct on the notification id.

If you see	It means
No Audit rows	The Audit types parameter is narrowed. Leave it empty for all types.
rows equals real_alerts	Only one fetch has run, or every alert closed immediately. Not an error.

2. Alert Triage

Why: this is the pack's main automation. It must reach a verdict from KOI's real vocabulary, and must not re-investigate the same alert over and over.

Do

1. Attach KOI - Alert Triage to a KOI alert, or run it on an existing one:

```
!setPlaybook playbookId="KOI - Alert Triage"
```

2. Open the war room.

Expect

- A KoiContext object carrying item id, marketplace, risk level and hostname.
- An investigation summary, then a verdict: Malicious, Suspicious or Benign.
- Low-risk items auto-close. Critical and high escalate. Medium and pending stay open for an analyst.

Why those three: the verdict keys on risk_level, which KOI sends in lowercase (critical / high / medium / low / pending), plus the independent catalog risk. A pending item is one KOI has not finished assessing, so it deliberately does not auto-close.

Then test the dedupe

Run triage twice against alerts sharing one notification id.

Expect the second run to stop early with a DUPLICATE entry naming the alert that already handled it.

If you see	It means
KoiContext empty, everything Suspicious	The alert carried no OCSF payload and no koi_context blob. Check the correlation rule passes the KOI fields through. This is the most common cause.
No DUPLICATE on a repeat	The notification id was empty. The gate is built to fall through to normal triage rather than swallow an alert.

3. Investigation Depth

Why: a verdict is only as good as what sits behind it, and an analyst approving a block needs the whole picture.

Do

1. Run KOI - Investigate Item with an item_id and marketplace.
2. Run KOI - Investigate Device with a device id.

Expect

- A KoiInvestigation summary with catalog risk and the AI summary.
- Organisation exposure: installs, endpoints affected, users.
- **Both governance counts — blocklisted_count and allowlisted_count.**
- For the device: everything installed on that host, plus a risky-items table.

Why both counts matter: zero blocklist hits alone does not mean "not governed". It can mean somebody explicitly allowed the item. Without the allowlist read, an allowlisted item stays a valid block candidate.

Enrichment is best-effort by design. If it comes back empty while the playbook stays green, confirm reachability with !koi-users-list limit=1 before assuming a content fault.

4. Gated Response

Why: this is the only state-changing action in the pack. It must never fire without a human.

Do

1. Run KOI - Block and Remediate with an item_id and marketplace.

Expect

- The run parks on an approval task.
- The approval shows catalog and org risk, exposure, remediation history, and the governance line — including a warning when the item is already allowlisted.
- Nothing reaches the blocklist until somebody approves.
- An item already on the blocklist short-circuits instead of being added twice.

This is the gate to re-run after any change to the response playbook. If it ever completes without parking, stop and investigate before using the pack in production.

5. Proactive Hunting

Why: finds risk that nobody alerted on.

Do

1. Run KOI - MCP Server Audit, directly or from a Job.
2. Run a few library hunts from docs/xql — start with H2.1 and H1.3.

Expect

- The audit lists MCP servers at or above the risk threshold.
- The hunts return findings KOI scored but nobody actioned.

Worth knowing	Detail
view=mcp_servers	Now selectable in the argument dropdown. It previously worked only when a playbook passed it literally.
view=all_items	Accepted by the API but returns nothing. Omit view entirely to query everything.
The library	26 hunts and 46 detections in docs/xql. Read QUERY_LIBRARY.md first. They read raw dataset columns, so they do not depend on the modeling rule.

6. Fleet Script Execution

Why: runs KOI deployment scripts across endpoints on a schedule, and must cover groups larger than the platform's 100-endpoint query cap.

Do

1. Create the JSON List Koi Script Runner, with tracker_list and rescan_interval_hours set per entry.
2. Run the Refresh job (Koi Unified - Refresh Tracker) **first**.
3. Then run the Scan job (Koi Unified - Script Runner).

Expect

- Refresh fills the tracker List with endpoint_id,last_scan rows — it creates the List for you.
- Scan dispatches to due, connected endpoints and stamps their last_scan.
- Offline, wrong-OS and not-in-inventory endpoints stay due and retry on the next run.
- A second Scan straight afterwards does nothing — everything is inside the rescan interval.

Why Refresh first: Scan reads the tracker. An empty tracker means nothing is due, and that run is a legitimate no-op that can easily look like a failure.

Outcomes that are not failures

Entry	What it means
SKIPPED	Nothing due and connected right now. Sends no email, by design.
STILL RUNNING	Dispatched, but polling ended before Action Center finished. The endpoints are already marked and the action completes on its own. Sends no email.

If neither job ever runs, confirm each Job is enabled and shows a next-run time. A newly created Job can land disabled.

7. Dashboard

Why: these numbers get quoted to other people.

Do

Open the KOI Alerts Dashboard.

Expect

- Alert counts far lower than raw row counts.
- Counts that stay stable as fetch cycles repeat.

Every Alerts widget dedupes on the notification id before counting, so it reports alerts rather than re-deliveries. If a widget's number climbs steadily while nothing new is happening in KOI, that widget is counting fetches.

Sign-off

One line of evidence per use case.

#	Use case	Evidence
1	Event collection	Both Alerts and Audit present; count_distinct(nid) well below raw rows
2	Alert triage	Verdict reached; low risk auto-closes; a repeat notification stops as DUPLICATE
3	Investigation depth	Investigation carries catalog risk and both governance counts
4	Gated response	Run parks on approval; blocklist untouched until approved
5	Proactive hunting	MCP audit returns servers at or above threshold; hunts run
6	Fleet script execution	Refresh fills the tracker; Scan marks only due and connected; re-run is a no-op
7	Dashboard	Alert counts stable across fetch cycles

All seven green means the pack is ready for production use.

Full detail is in the customer guide (KOI_Integration_Customer_Guide_v1.4.0). Deeper diagnostics are in the troubleshooting guide.